

Team Delivery Package: Evaluation & Insights Engine

Enterprise Milestone-Based Technical Specifications

MODULE NAME:	Team 6: Evaluation & Insights Engine
GIT BRANCH:	feature/evaluation-insights
LEAD MEMBERS:	Youssef Al-komi, Sohad Abd El-Mohsen
WORKLOAD TIME:	2 weeks
KICKOFF DATE:	11 July 2026
CODE FREEZE:	23 July 2026
VERSION:	v1.2.0

1. Cover Page & Metadata Summary

This delivery package establishes the contract specifications for the **Team 6: Evaluation & Insights Engine**.

Milestone	Target Date	Status	Description
Project Kickoff	11 July 2026	COMPLETED	Milestone scope locked & repository structured.
Code Freeze	23 July 2026	IN PROGRESS	All functional packages and tests locked.
Integration Phase	24 July 2026	PLANNED	Multi-team pipeline joins and bug resolution.
Final Presentation	25 July 2026	PLANNED	Final delivery and platform presentation.

2. Team Overview & Assignments

Resource Details

Lead Team Members: **Youssef Al-komi, Sohad Abd El-Mohsen**

Branch Space: ``feature/evaluation-insights``

Role Assignment: **Evaluation Engineers & Insight Architects**

Difficulty Rating: **Medium** | Priority Index: **High** | Est. Workload: **2 weeks**

3. Business & Technical Goals

Business Objective

Translate complex metrics into actionable insights for non-technical users.

Technical Objective

Calculate evaluation indices (Accuracy, R²) and design a rule-based insight engine.

Position inside AutoAnalyst AI

Evaluation and Insight Generation block.

4. System Architecture & folder layout

Assigned folder Tree Structure

```
src/autoanalyst/  
  evaluation/  
    __init__.py  
    evaluator.py  
  insights/  
    __init__.py  
    insight_generator.py
```

Target Code Modules

Your codebase modifications belong inside: `src/autoanalyst/evaluation/evaluator.py`,
`src/autoanalyst/insights/insight\_generator.py`.

Functional Code Interface Specifications

```
# src/autoanalyst/evaluation/evaluator.py
from typing import Any

def evaluate_classification(y_true, y_pred) -> dict[str, Any]:
    """Calculates accuracy and returns scikit-learn classification report."""
    pass

# src/autoanalyst/insights/insight_generator.py
def generate_dataset_insights(df: pd.DataFrame) -> list[str]:
    """Generates descriptive textual insights of dataset anomalies."""
    pass
```

API Specification

```
evaluate_regression(y_true: Array, y_pred: Array) -> Dict[str, float]
Output Return Schema:
{
    "mae": float,
    "mse": float,
    "r2": float
}
```

5. Git Workflow & Code Integration Policy

Branching Workflow

All developers pull and write inside `feature/evaluation-insights` created from the latest `develop` branch.

Pull Request Requirements

- 1. Merge latest develop branch changes into features before raising PRs.
- 2. Assign team lead reviews and resolve conflicts.
- 3. A minimum of one approved review check is required.

6. Quality Standards & Guidelines

Format & Lint Rules

All variables, methods, and file layouts must use lower_case snake_case parameters. Code comments must be in English.

Error Checking & Inputers

All functional blocks should be guarded by custom exception checks (raise ValueError/KeyError).

Logging Rules

Incorporate logging warnings rather than std print statements.

7. Expected Deliverables & Verification

Required Files list

`src/autoanalyst/evaluation/evaluator.py, src/autoanalyst/insights/insight\_generator.py`

Verification Code Assertions

Unit test validations are situated in: `tests/test\_evaluation.py, tests/test\_insights.py`.

Pipeline integration assertions belong in: `tests/test\_pipeline.py`.

IMPORTANT GUIDELINE

All functional checks must compile successfully with zero syntax warnings before code freeze.

8. Definition of Done Checklist

- [] All functions compile cleanly with PEP 8 checks.
- [] Unit tests cover all key methods and functions.
- [] Try-except checks isolate external dataframe issues safely.
- [] The PR has been reviewed and verified by Team 1.

9. Common Mistakes & Troubleshooting

Pitfalls to Avoid

- Running regression metric equations on discrete class label strings.
- Hardcoding fixed column names (e.g. assuming 'age' is always present) inside insights.
- Introducing division-by-zero errors when calculating recall for zero positive samples.